CHEATSHEET DE RÉVISION COMPLÈTE IEV – ISIA (2026–2027) • Polytech Lille

Plongée dans les environnements virtuels Python

Support de référence : Juliette Achddou & Bastien Cazaux • Couvre l'intégralité du cours (59 slides & 13 activités TP)

▶ SITE WEB & SIMULATEUR CLI
Pratiquez les commandes & les 13 Mini-TPs dans le terminal simulé :
[URL À REMPLACER : https://iev-cheatsheet.vercel.app]

Sommaire : **Page 1 :** Ch.1 Shell CLI, Arborescence, PATH & Inception • Ch.2 Packages, PyPI, pip, Graphe DAG, SemVer & Conflits — **Page 2 :** Ch.3 venv, requirements.txt, virtualenv • Ch.4 pyproject.toml, lock, Poetry & uv • Ch.5 Conda, Containers, VMs & Décision

CHAPITRE 1 — RAPPELS SUR LES SHELLS CLI, SYSTÈME DE FICHIERS, VARIABLES & PATH

Slides 6 à 18 (Pages 9–26)

Shell vs Terminal vs Console (Slide 7) : Le **shell** (bash, zsh) est le programme qui traite les commandes et renvoie un résultat (interface de l'OS). Le **terminal** est le programme d'interface qui exécute un shell. La **console** est un type particulier de terminal.

Commande / Syntaxe	Effet, Options & Détails du cours
`man ls` | `ls --help`	Manuel d'aide **hors-ligne** (disponible pendant les examens déconnectés !).
`pwd` | `pwd --version`	*Print Working Directory* : affiche le dossier courant (auteur : **Jim Meyering**, GNU coreutils).

`cd /home/bcazaux/mes-secrets`
`cd ~/Documents/mes-examens/`
`cd ../../bcazaux/mes-secrets`
	• **Chemin de la racine (absolu) :** part de / (slash sous Linux).
	• **Chemin partant du home :** part de ~ (/home/user).
	• **Chemin relatif :** part du dossier courant (`..` remonte au dossier parent).

`printenv` | `env`	Affiche toutes les variables d'environnement définies dans la session courante.
`echo $PATH`	Affiche une variable d'environnement (utiliser obligatoirement le préfixe $).
`export PATH="CH1:CH2:CH3"`	Définit/modifie une variable pour la session. PATH est une liste de dossiers séparés par `:` parcourus dans l'ordre par le shell pour trouver les applications (`cowsay`, `sl`).
`source ~/.bashrc`	Recharge immédiatement le fichier `~/.bashrc` dans la session courante sans rouvrir de terminal.

`bash` | `bash --norc`
`echo $SHLVL` | `exit`
`ps -f --forest`
	Inception of Bash (Slides 16–17) : `bash` ouvre un sous-shell ; `exit` le ferme. $SHLVL donne la profondeur d'imbrication ; `ps -f --forest` affiche l'arbre d'imbrication des shells.

Portée temporelle d'une modification du PATH (Slide 14 — TP sl) :

Action réalisée	Passé (Déjà ouvert)	Présent (En cours)	Futur (Nouveau)
1. `export PATH=...` dans le terminal	✗	✓	✗
2. `export PATH=...` dans `.bashrc`	✗	✗	✓
3. Dans `.bashrc` + `source ~/.bashrc`	✗	✓	✓

```
# Pourquoi bash --norc conserve le prompt PS1 ? (Slide 17)
pseudo$ export PS1="[A] > "
[14:32] > bash        # Lit ~/.bashrc -> réinitialise PS1 (pseudo$)
pseudo$ exit
[14:32] > bash --norc # Ignore ~/.bashrc -> garde
[14:32] >
```

⚠️ **2 Pièges fréquents :**
1. Toujours écrire `export PATH="$PATH:/mon/dossier"` (sans $PATH:, vous écrasez les chemins système et perdez `ls`, `python3`...).
2. Sous Linux, respectez la casse : écrivez toujours **`python3`** en minuscules (jamais `Python3`).

CHAPITRE 2 — PACKAGES PYTHON, PYPI, PIP, GRAPHE DES DÉPENDANCES, SEMVER & CONFLITS

Slides 18 à 29 (Pages 27–47)

PyPI (Python Package Index — Slide 22) : Plateforme en ligne hébergeant les bibliothèques Python :
• **Paquets `sdist` (`*.tar.gz`) :** code source compressé nécessitant une compilation locale.
• **Paquets `wheel` (`*.whl`) :** fichier binaire prêt à l'emploi (installation rapide sans compilation).

Commande pip	Rôle & Distinction à connaître
`pip install black` `python3 -m pip install black`	Installe un paquet et ses dépendances. *Note :* Sur le système de l'école (Slide 21), vous n'avez pas les droits admin pour installer globalement avec `pip` !
`pip --version` | `pip help`	Affiche la version de `pip`, son chemin de stockage exact et la version de Python associée.
`pip list` vs `pip freeze`	• **`pip list` :** tableau formaté pour la lecture humaine. • **`pip freeze` :** format strict `paquet==version` destiné à `requirements.txt`.

Graphe des dépendances (DAG sans cycle) & Tri topologique (Slides 23–24) :
Un package A dépend de B si A a besoin de B. Ce graphe **ne peut pas contenir de cycle**. Pour trouver l'ordre d'installation, `pip` réalise un **tri topologique** :
`pkg_e` ⟶ `pkg_f` ⟶ `pkg_b` ⟶ `pkg_a` ⟶ `pkg_c` ⟶ `pkg_d` ⟶ `myapp`
Conflit en diamant & Enfer des dépendances (Slides 27–28) : Dans le TP 6, `myappconf` dépend de pkg_a (qui exige `pkg_c < 2`) ET de pkg_b (qui exige `pkg_c >= 2`) ⟶▶ installation impossible !

```
Semantic Versioning (SemVer – Slide 25) : MAJEUR.MINEUR.CORRECTIF
• CORRECTIF (1.10.7 ⟶ 1.10.8) : Corrections d'anomalies rétrocompatibles
• MINEUR    (1.10.8 ⟶ 1.11.0) : Ajouts de fonctionnalités rétrocompatibles
• MAJEUR    (1.11.0 ⟶ 2.0.0)  : Changements NON rétrocompatibles
```

Délimitation des versions des dépendances (Slide 26) :

Version	==1.3.1	!=1.2.1	>1.2.4	<=1.2	~=1.2	~=1.1.1
numpy 1.1.0	■	●	■	●	■	■
numpy 1.1.1 / 1.1.2	■	●	■	●	■	●
numpy 1.2.1	■	■	■	●	●	■
numpy 1.2.4	■	●	■	●	●	■
numpy 1.3.1	●	●	●	■	●	■
numpy 2.0.3 / 2.1.x / 3.1.0	■	●	●	■	■	■

Opérateur compatible ~= : `~=1.2` équivaut à `>=1.2, <2.0` | `~=1.1.1` équivaut à `>=1.1.1, <1.2.0`.

CHAPITRE 3 — ENVIRONNEMENTS VIRTUELS CLASSIQUES : VENV, REQUIREMENTS.TXT & VIRTUALENV

Slides 30 à 39 (Pages 48–78) • Analogie : « La Lunch Box »

Définition (Slide 30) : Distribution Python **isolée et autonome** contenant une copie de Python + un dossier `site-packages` dédié. Permet d'installer des paquets avec `pip` sur les sessions de l'école et d'éviter les conflits entre projets (env1 avec `numpy 1.0.0`, env2 avec `numpy 2.3.3`).

Commande	Rôle & Pratique (TP 7 à 10)
`python3 -m venv mon_env`	Crée un environnement virtuel avec `venv` (module standard inclus depuis Python 3.3, même version Python que celle appelée).
`source mon_env/bin/activate`	Active l'environnement dans le shell courant. Pour le désactiver : `deactivate`.
`pip freeze > requirements.txt` `pip install -r requirements.txt`	Exporte les dépendances à la racine du projet, puis les réinstalle en une seule fois dans un nouvel environnement (ex: pour `pokestat.py`).
`virtualenv -p python3.11 myenv`	Crée un environnement avec **n'importe quelle version de Python** (2.x, 3.11...) + **seeder ultra-rapide** depuis un cache local (`pip`, `setuptools`, `wheel`).
`cowsay -t "Bonjour !" -c tux | lolcat -a`	Chaîne la sortie de `cowsay` (personnages disponibles via `cowsay -h`) dans `lolcat` grâce au pipe `|`.

Les 11 propriétés d'une « bonne Lunch Box » (Slide 31) :

1. **Isolation** des dépendances
2. **Bonne pratique** pro
3. **Flexibilité** des modèles
4. **Portabilité** du projet
5. **Simplicité** de gestion
6. **Frugalité** (léger : ~150 Mo)
7. **Compatibilité** multi-versions
8. **Reproductibilité** (recette)
9. **Facilité CI/CD** (standardisé)
10. **Sécurité** (test sans risque)
11. **Nettoyage facile** (jetable/remplaçable sans perdre son frigo)

Écosystème `virtualenv` (Slide 36) :

- **Extensible :** dépôt PyPI privé, préinstallation automatique (`pytest`, `black`).
- **`virtualenvwrapper` :** centralise les environnements (`mkvirtualenv`, `workon`).
- **`tox` :** crée plusieurs environnements et lance automatiquement les tests.

CHAPITRE 4 — GESTION MODERNE : PYPROJECT.TOML, LOCK, POETRY (= CHEFFE DE CUISINE) & UV

Slides 40 à 51 (Pages 79–111)

Pourquoi `requirements.txt` ne suffit pas ? (Slide 42)

1. Pour `matplotlib` sans version, laquelle mon collègue a-t-il utilisée ?
2. Faut-il figer `numpy==1.26.4` si d'autres versions sortent ?
3. Quelle version de **Python** est utilisée ?
4. Quelle est la version / auteur du projet ?

1. `pyproject.toml` = LE SOUHAIT (Slides 43–46)

Format **TOML** (*Tom's Obvious, Minimal Language*). Décrit nos envies et contraintes : `[build-system]`, `[project]` (`name`, `version`, `authors`, `requires-python = ">=3.9"`, `dependencies`), `[project.optional-dependencies]` (`dev`, `docs`), `[tool.*]`.

2. `poetry.lock` = LA RÉALITÉ FIGÉE (Slide 46)

Résout et fige une version précise et compatible de tous les paquets. Garantit la reproductibilité (CI, déploiement) : *la photo exacte de ce qui marche*.

3. Env. Virtuel = L'EXÉCUTION concrète du lock.

Étape (Slide 47)	Commandes Poetry	Effet dans le flux Poetry
1. Démarrer	`poetry new mon_projet` | `poetry init`	Crée ou initialise `pyproject.toml`
2. Dépendances	`poetry add requests`	Ajoute à `pyproject.toml`, résout le lock et installe
3. Verrouillage	`poetry lock` | `install` | `update`	`lock` résout ; `install` installe le lock ; `update` met à jour
4. Dev & Run	`poetry shell` | `poetry run python -m ...`	Active le venv ou exécute une commande dedans
5 & 6. Dist/Pub	`poetry build` | `poetry publish -u ... -p ...`	Construit `dist/` (`sdist` / `wheel`) et publie sur PyPI

Les concurrents de Poetry & la révolution uv (Slide 50) :

Outil	Py	Pkgs	Build	Upload	Venv	Lock
`venv` / `virtualenv`	no	no	no	no	yes	no
`pip`	no	yes	yes	no	no	yes
Pipenv	no	yes	no	no	yes	yes
Poetry / PDM / Flit	no	yes	yes	yes	yes	yes
pyenv	yes	no	no	no	no	no
uv (Rust) / Pyflow	yes	yes	yes	yes	yes	yes

⚡ **Benchmark (cache chaud) :** `uv` (**0.06s**) vs `Poetry` (0.99s) vs `PDM` (1.90s) vs `pip-sync` (4.63s).

CHAPITRE 5 — AU-DELÀ DE PYTHON : CONDA (= CHEFFE À DOMICILE), CONTAINERS, VMS & ISOLATION

Slides 52 à 58 (Pages 112–125)

5 Limites des venvs classiques & Poetry (Slide 52) : 1. Pas de gestion des **dépendances système** (ex: `libgdal`, `proj`) • 2. Poetry reste centré sur Python • 3. Erreurs de compilation (compilateur/entêtes C absents lors du `pip install`) • 4. Pas de contrôle natif sur l'installation de Python • 5. Portabilité inter-machines limitée.

Commandes Conda (Slides 53–55)	Fonctionnement & Rôle
`/usr/localTP/anaconda3/bin/conda init`	Initialise Conda dans le shell (à faire une première fois).
`A. conda env list`	Liste tous les environnements Conda installés (ex: `base`, `mongoenv`).
`B. conda create -n myenv python=3.12`	Télécharge Python 3.12 + paquets **binaires précompilés** depuis `anaconda.org` selon `OS + arch + contraintes` (`linux-64`, `osx-64`, `win-64`).
`C. conda activate myenv` `D. conda install manim`	Active l'environnement et installe un paquet avec ses dépendances système natives (C/C++/FFmpeg/Cairo).
`E. conda deactivate` | `F. conda remove -n myenv --all`	Désactive l'environnement courant (E) ou le supprime totalement (F).

Pyramide de l'isolation & Poids mesuré (Python 3.9 + matplotlib — Slide 58) :

Niveau & Outils	Type d'isolation	Taille mesurée
1. Virtual Envs (`venv`, `Poetry`)	**Logique** (dépendances Python)	**144 à 178 Mo**
2. Conda Envs (`Conda`, `Mamba`)	**Logicielle** (dépendances + binaires)	**370 à 1900 Mo**
3. Docker Containers (`Docker`, `Podman`)	**Système** (noyau OS partagé)	**215 à 1134 Mo**
4. Virtual Machines (`VirtualBox`, `Vagrant`)	**OS & ressources virtuelles** (Guest OS)	Plusieurs Go
5. Bare Metal (Machine distincte)	**Physique** (aucun partage matériel)	Machine dédiée

Containers vs VMs (Slide 57) :

- **Containers :** Infra ⟶ Host OS ⟶ **Container Engine** ⟶ [Bin/Lib + App] (Noyau partagé)
- **VMs :** Infra ⟶ **Hypervisor** ⟶ [**Guest OS complet** + Bin/Lib + App]